

CHAPTER 8.

모바일 웹페이지



모바일 웹 디자인

- 요즘은 모바일이 대세이다 보니, 스마트폰이나 태블릿에서 웹사이트에 접속하는 경우가 많이 발생한다. 일반적으로 스마트폰이나 태블릿은 화면의 크기가 작기 때문에, PC용으로 제작된 웹사이트를 스마트폰으로 보면 글자가 너무 작아진다.

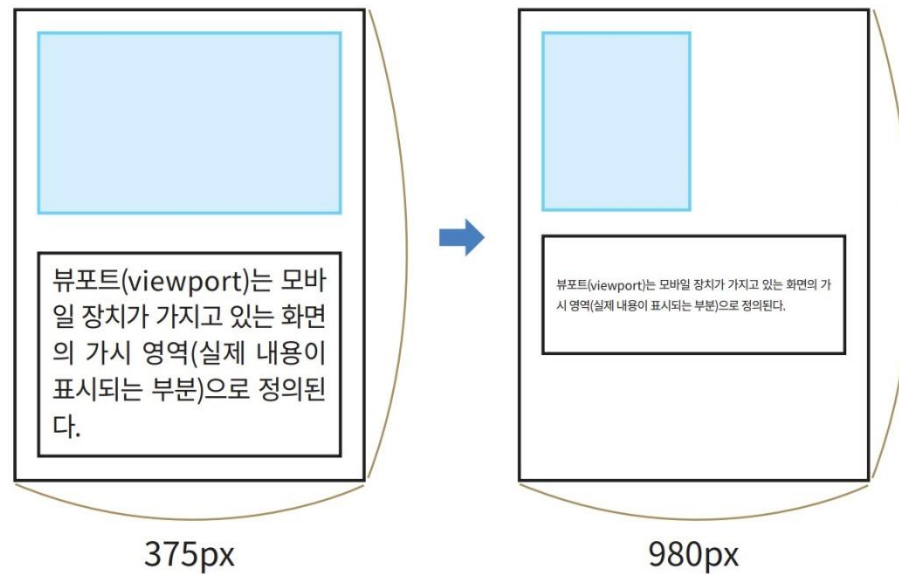


모바일을 위한 2가지 방법

1. 포털 사이트나 쇼핑몰 사이트에서는 **m.naver.com**와 같은 모바일 버전 사이트를 만들어서 유지하였다. 이것을 “적응형 웹”이라고도 한다.
 1. 서버에서 클라이언트가 모바일 장치인지 **PC**인지를 파악해서, 접속하는 사이트를 아예 다르게 하는 것이다.
 2. 하지만 이 방법을 사용하려면 웹사이트를 두 가지의 별도 버전으로 개발하여야 했다.
2. 하나의 웹페이지를 가지고 다양한 화면 크기에 맞춰서 변형할 수는 없을까?
이런 이유로 등장한 기술이 반응형 웹 디자인(**responsive web design: RWD**)이다.

뷰포트

- 뷰포트(viewport)는 모바일 장치가 가지고 있는 화면의 가시 영역(실제 내용이 표시되는 부분)으로 정의된다.
- 아이폰은 기본적으로 980px 크기의 뷰포트를 가정하므로 980px 크기로 표시하려고 하는 것이다. 이것을 방지하려면 개발자가 웹페이지에 장치의 해상도로 뷰포트를 지정하는 문장을 추가해야 한다.



애플사의 가상 뷰포트

- 스마트폰이 처음으로 등장했을 때는 화면의 해상도가 너무 낮아서(아이폰 3GS의 경우, 320×480), PC용으로 제작된 웹페이지들이 한 화면에 담기지 못하고 넘쳐 버렸다.
- 애플은 이 문제를 해결하기 위하여 웹페이지를 실제 스마트폰 해상도로 렌더링하지 않았고, 980픽셀 크기의 화면이라고 가정하고 웹페이지를 렌더링한 후에, 이를 축소하여 320픽셀의 화면에 보여주었다.
- 사용자는 터치를 이용하여 원하는 부분을 확대하여 볼 수 있었다. 이것이 바로 가상 뷰포트의 시작이었다.

애플사의 가상 뷰포트



뷰포트 지정하기

- 뷰포트는 다음과 같은 형식으로 `<meta>` 태그를 이용하여 웹페이지의 헤드 부분에서 지정한다.

`<meta name="viewport" content="width=device-width, initial-scale=1">`

뷰포트의 너비는 장치 너비로 한다. ←

초기 화면 배율은 1로 지정한다. ←

속성	설명	값
width	뷰포트 너비	device-width(장치 너비) 또는 구체적인 크기
height	뷰포트 높이	device-height(장치 높이) 또는 구체적인 크기
user-scalable	확대 축소 가능 여부	yes 또는 no
initial-scale	초기 화면 배율	1부터 10까지

반응형 웹디자인

- 반응형 웹페이지 디자인(responsive web design: RWD) - 웹페이지가 사용자의 환경을 읽어서 적절하게 반응하는 페이지
- By 웹디자이너 에단 마르코트(Ethan Marcotte)



반응형 웹디자인의 3가지 기법

- 첫 번째 아이디어는 미디어 쿼리(media query)이다. 미디어 쿼리를 사용하면 장치가 가지고 있는 실제 화면의 크기를 알아낼 수 있고, 화면 크기에 따라 화면의 레이아웃을 변경할 수 있다.
- 두 번째 아이디어는 유동 레이아웃(fluid layout)이다. 기본적인 개념은 화면의 레이아웃을 설계할 때 픽셀과 같은 절대적인 수치보다는 퍼센트(%)와 같은 상대적인 수치를 사용하는 것이다.
- 세 번째 아이디어는 유동 이미지(fluid image)이다. 이미지를 장치의 크기에 맞추는 기술이다.

RWD 방법 1. 미디어 쿼리

- 2009년에 도입된 미디어 쿼리를 이용하여 우리는 장치의 해상도를 파악할 수 있다. 장치의 해상도에 따라서 서로 다른 스타일 시트를 적용하는 것이 핵심이다.

```
@media screen and (min-width: 800px) and (max-width: 1200px) {  
  ...  
}
```

미디어 유형. screen, print, tv, projection 등이 있다.

최소 너비가 800px은 되어야 한다.

최대 너비가 1200px이다.

미디어 쿼리

속성	설명
width	뷰포트의 너비
height	뷰포트의 높이
min-width	뷰포트의 최소 너비
min-height	뷰포트의 최소 높이
max-width	뷰포트의 최대 너비
max-height	뷰포트의 최대 높이

속성	설명
device-width	장치의 너비
device-height	장치의 높이
min-device-width	장치의 최소 너비
min-device-height	장치의 최소 높이
max-device-width	장치의 최대 너비
max-device-height	장치의 최대 높이

화면의 방향 알아내기

- 화면의 방향에 따라서 서로 다른 스타일을 적용하는 것도 가능하다. 이것은 미디어 쿼리의 **orientation** 속성을 이용한다. **orientation** 속성이 **portrait**이면 세로 모드이다. **landscape**이면 가로 모드이다.

```
@media screen and (min-device-width: 375px) and (orientation: landscape) {  
    ...  
}
```

중단점

- CSS에 여러 미디어 쿼리를 추가하여 화면 크기에 맞는 여러 개의 레이아웃을 지정할 수 있다. 화면의 크기에 따라 스타일이 달라지는 분기점을 중단점(breakpoint)이라고 한다



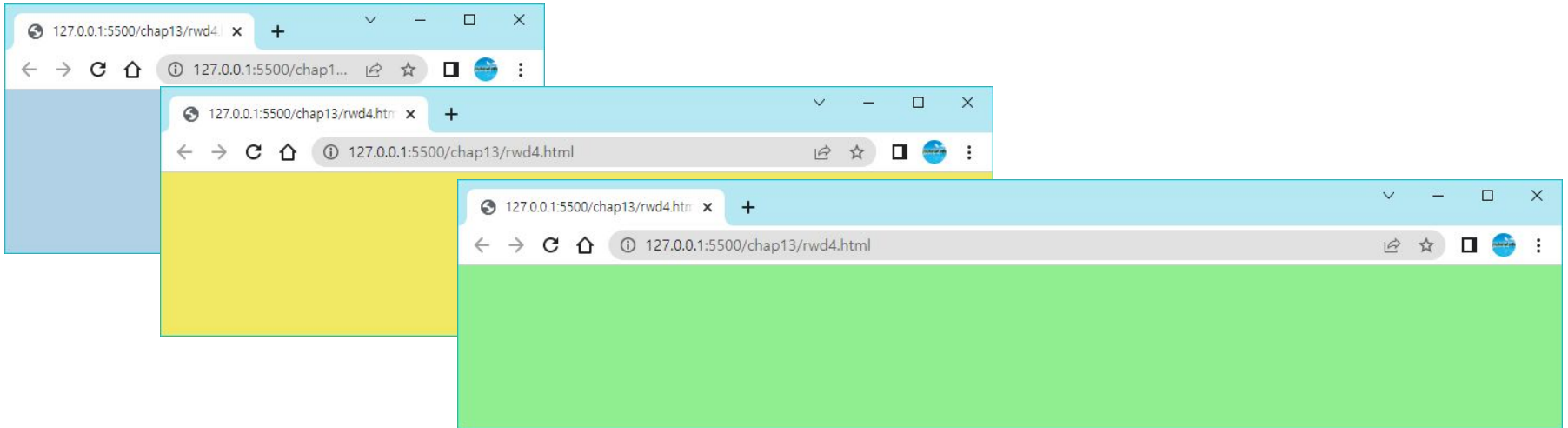
모바일 퍼스트

- 미디어 쿼리를 사용할 때 일반적인 접근법은 일단 스마트폰과 같이, 좁은 화면을 가지는 장치에 대하여 단일 열 레이아웃을 만든 후에, 화면이 충분히 크면 다단 레이아웃을 구현하는 경우가 많다.
- 즉 스마트폰에 대한 레이아웃이 기본이 된다. 이것을 흔히 모바일 퍼스트 (**mobile first**) 디자인이라고 한다.
- 모바일을 PC보다 우선시 하겠다는 정책이다.

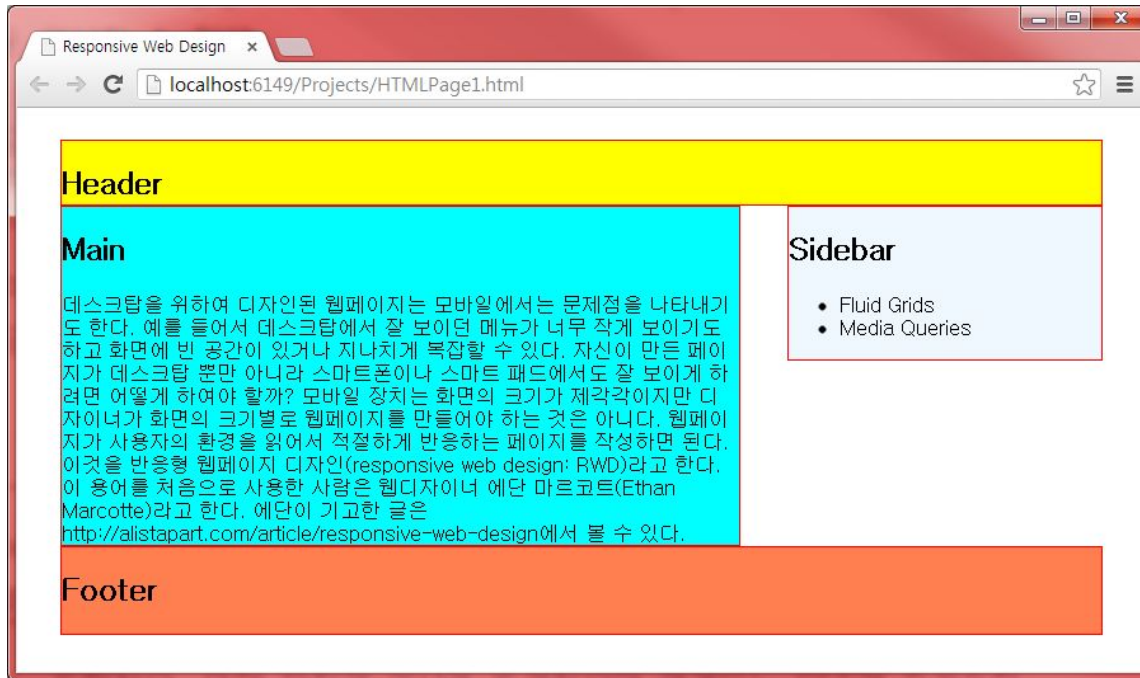
예제

```
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <style>
    body {
      background-color: lightgreen;
    }
    @media screen and (max-width: 1024px) {
      body {
        background-color: rgb(241, 232, 99);
      }
    }
    @media screen and (max-width: 768px) {
      body {
        background-color: rgb(177, 210, 230);
      }
    }
  </style>
</head>
<body></body>
</html>
```

실행 결과



RWD 레이아웃 예제



웹브라우저에서 실행하기

예제

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Responsive Web Design</title>
  <style>
    #page {
      padding: 5px;
      width: 960px;
      margin: 20px auto;
    }
    #header { height: 50px; }
    #main { width: 600px; float: left; }
    #sidebar { width: 300px; float: right; }
    #footer { clear: both; }
```

예제

```
@media screen and (max-width: 980px) {  
  #page { width: 94%; }  
  #main { width: 65%; }  
  #sidebar { width: 30%; }  
}
```

```
@media screen and (max-width: 700px) {  
  #main { width: auto; float: none; }  
  #sidebar { width: auto; float: none; }  
}
```

```
@media screen and (max-width: 480px) {  
  #header { height: auto; }  
  h2 { font-size: 24px; }  
  #sidebar { display: none; }  
}
```

예제

```
#header, #main, #sidebar, #footer {  
    border: solid 1px red;  
}  
  
#header {          background-color: yellow;      }  
#sidebar {         background-color: aliceblue;    }  
#main {            background-color: aqua;         }  
#footer {          background-color: coral;        }  
</style>  
</head>
```

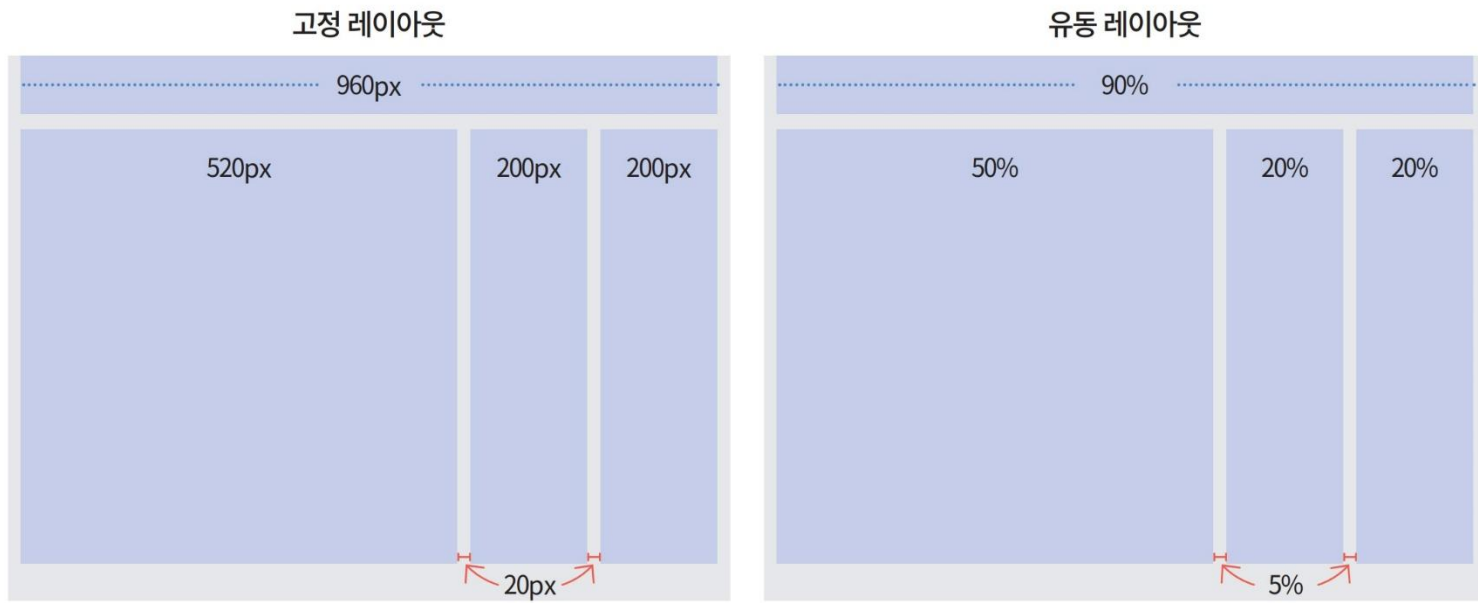
예제

```
<body>
  <div id="page">
    <div id="header">
      <h2>Header</h2>
    </div>
    <div id="main">
      <h2>Main</h2>
      ...
    </div>
    <div id="sidebar">
      <h2>Sidebar</h2>
      <ul>
        <li>Fluid Grids</li>
        <li>Media Queries</li>
      </ul>
    </div>
    <div id="footer">
      <h2>Footer</h2>
    </div>
  </div>
</body>
</html>
```

에 단이 기고한 글은 <http://alistapart.com/article/responsive-web-design>에서 볼 수 있다.

RWD 방법2. 유동 레이아웃

- RWD는 일반적으로 화면의 크기에 따라 레이아웃만 변경하는 것이 아니라, 유동 레이아웃(fluid layout, liquid layout)을 기반으로 구축된다.
- 유동 레이아웃은 레이아웃할 때 고정된 크기를 사용하는 것이 아니고 백분율과 같은 상대적인 단위를 사용하는 레이아웃 방법이다.



유동 레이아웃

- RWD의 초기에 유동 레이아웃을 수행하는 유일한 옵션은 **float**를 사용하는 것이었다. 유동 레이아웃은 각 요소에 백분율 너비를 부여하고, 레이아웃 전체에서 합계가 **100%**를 넘지 않도록 함으로써 달성되었다.
- 하지만 현대적인 레이아웃 방법인 **flexbox**나 **grid** 레이아웃은 기본적으로 반응적이다. 이 방법을 사용하는 것이 **float** 속성과 백분율 크기를 사용하는 방법보다 더 편리하다.

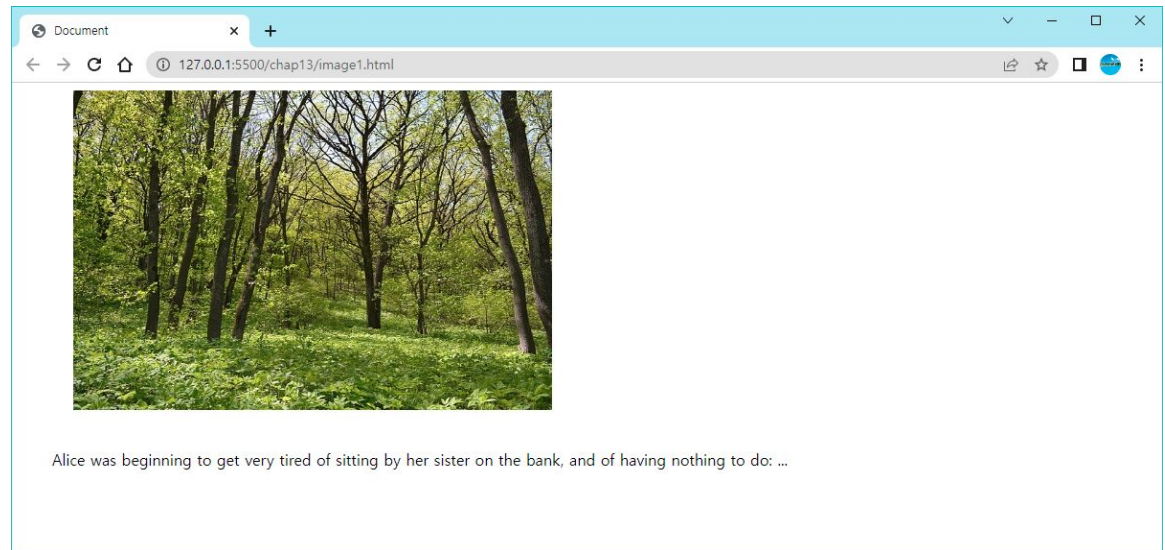
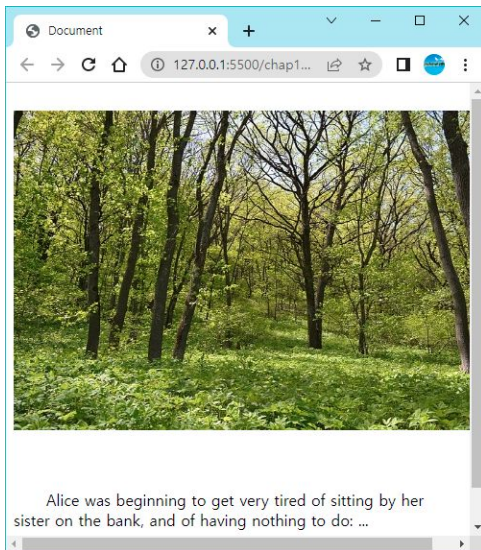
RWD 방법 3. 반응형 이미지

- RWD에서는 이미지를 표시할 때도 주의하여야 한다. 예를 들어 그림과 텍스트가 들어 있는 HTML 문서가 있다고 하자. 일단 이미지의 크기를 고정시키면 어떤 문제가 발생하는지 살펴보자.

```
<!DOCTYPE html>
<html lang="ko">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
  <style>
    .container { width: 640px; }
  </style>
</head>
<body>
  
  <div class="container">
    <p>
      Alice was beginning to get very tired of sitting by her sister on the bank, and of having
      nothing to do: ...
    </p>
  </div>
</body>
</html>
```


실행 결과

- 그림이나 컨테이너의 크기를 **640px**와 같이 고정 크기로 설정하면 브라우저의 너비가 작아지면 그림이나 텍스트가 잘리기도 하고, 반대로 브라우저의 너비가 커지면 빈 공간이 많이 발생한다.



반응형 이미지

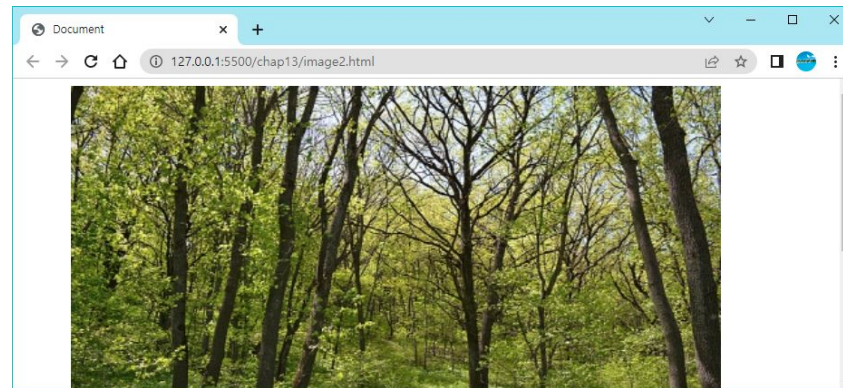
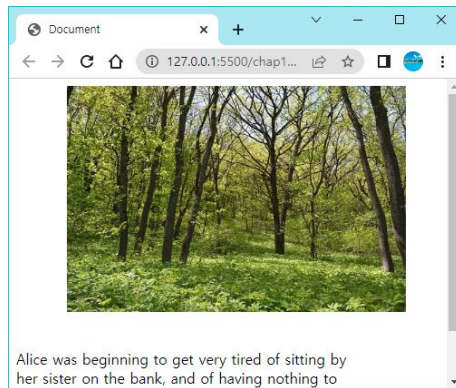
- 그림이나 컨테이너의 크기를 자동으로 변경하려면 컨테이너나 그림의 크기를 퍼센트 단위로 설정하는 것이 좋다.

```
...
<head>
  <meta charset="UTF-8">
  <title>Document</title>
  <style>
    .container { width: 80%; }
    img {
      width: 80%;
      height: auto;
    }

  </style>
</head>
<body>
  
  <div class="container">
  ...
```

실행 결과

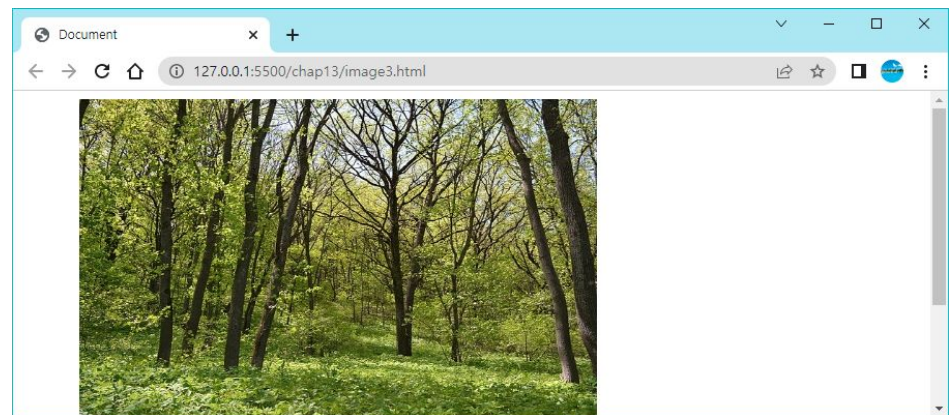
- 하지만 위와 같이 설정하면, 이미지는 원래 크기보다 크게 확대될 수 있다.



반응형 이미지

- 더 나은 솔루션은 `max-width` 속성을 사용하는 것이다. `max-width` 속성이 80%로 설정된 경우, 이미지는 원래 크기의 80%보다 확대되지는 않는다.

```
img {  
  max-width: 80%;  
  height: auto;  
}
```



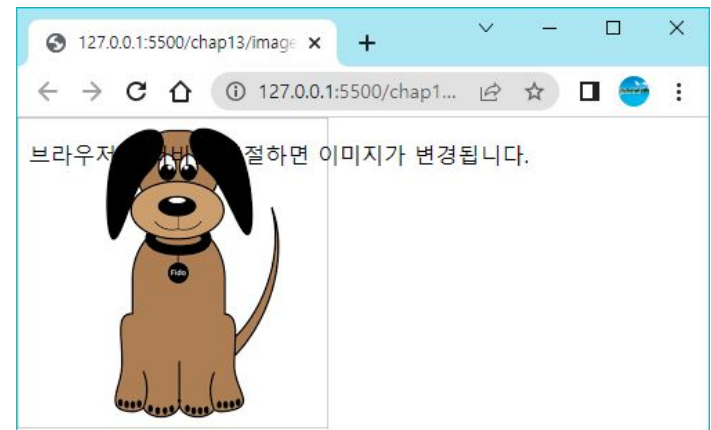
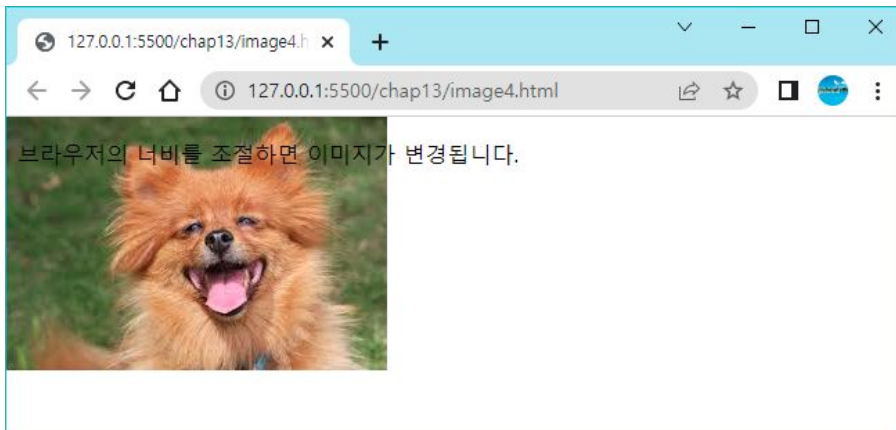
장치에 맞는 이미지 제공

- 큰 이미지를 로딩하는 것은 통신 대역폭을 낭비하는 것이다. 이 문제를 미디어 쿼리로 해결할 수 있다. 미디어 쿼리를 통하여 장치의 특성을 파악하여 적절한 이미지를 지정할 수 있다.

```
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <style>
    body {
      background-repeat: no-repeat;
      background-image: url('small.png');
    }
    @media screen and (min-width: 600px) {
      body {
        background-image: url('normal.jpg');
      }
    }
  </style>
</head>
<body>
  <p> 브라우저의 너비를 조절하면 이미지가 변경됩니다. </p>
</body>
</html>
```

장치에 맞는 이미지 제공

- 그림의 크기에 따라 다른 이미지를 로딩할 수 있다.



<picture> 요소 사용

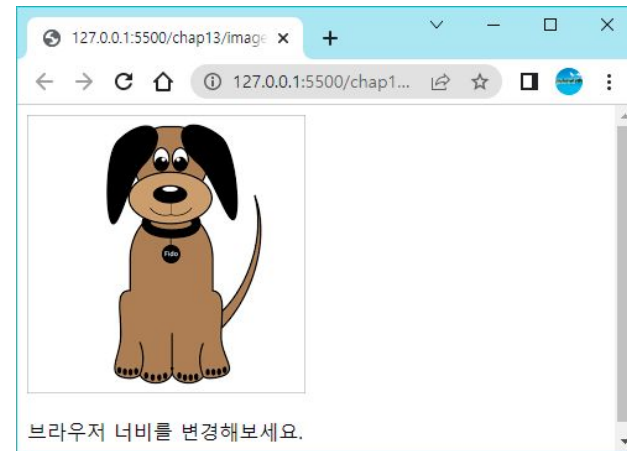
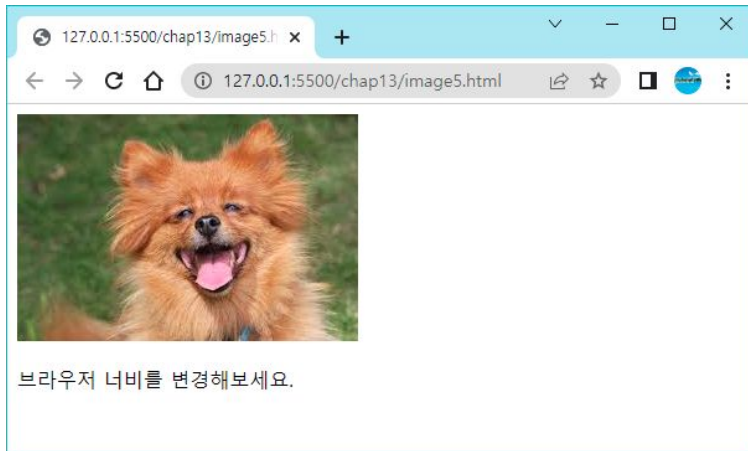
- HTML <picture>요소를 사용하면 웹 개발자가 이미지 리소스를 보다 유연하게 지정할 수 있다. <picture> 요소의 가장 일반적인 용도는 반응형 디자인에 사용되는 이미지에 사용된다.

```
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
  <picture>
    <source srcset="small.png" media="(max-width: 600px)">

    <source srcset="normal.jpg">

    
  </picture>
  <p>브라우저 너비를 변경해보세요.</p>
</body>
</html>
```

실행 결과



새로운 CSS. grid 레이아웃

- 그리드 레이아웃은 가로 방향의 행(row)과 세로 방향의 행(column)으로 구성된다.
- 각 그리드에 항목을 배치하여 원하는 형태의 레이아웃을 만든다.

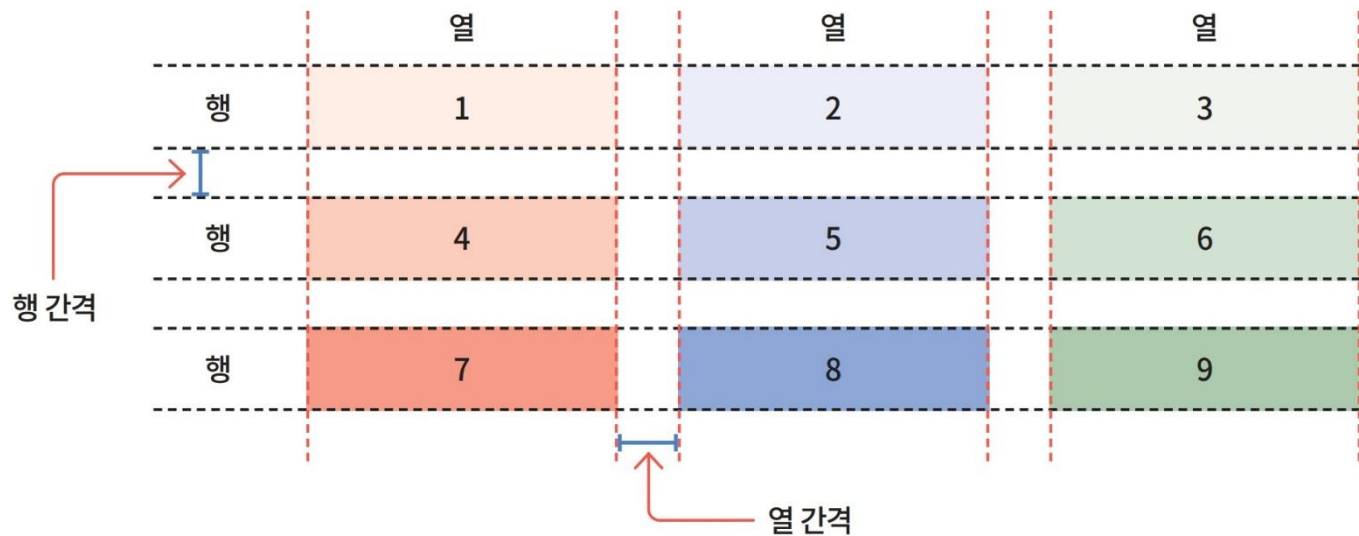
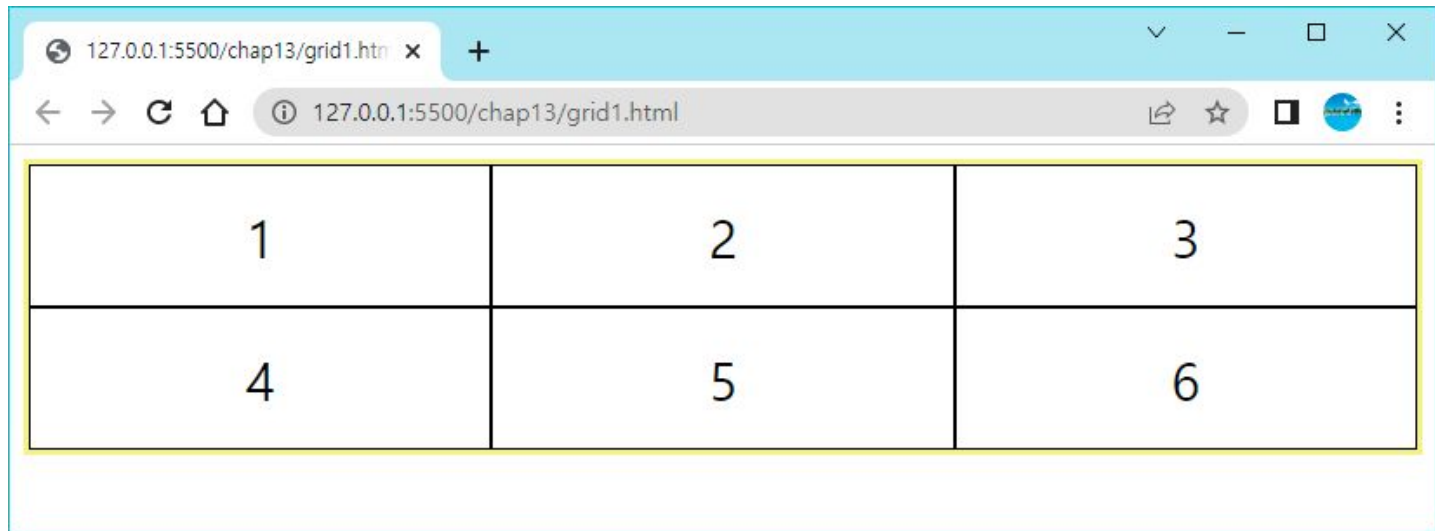


그림13-1 그리드 레이아웃

예제

- 그리드 레이아웃을 이용하여 다음과 같은 형태의 웹페이지를 표시하는 소스를 보자.



A screenshot of a web browser window displaying a 2x3 grid layout. The browser's address bar shows the URL `127.0.0.1:5500/chap13/grid1.html`. The grid consists of six cells, each containing a number from 1 to 6, arranged in two rows and three columns. The grid is highlighted with a yellow border.

1	2	3
4	5	6

Sol.

```
<!DOCTYPE html>
<html>
<head>
  <style>
    .container {
      display: grid;                // ①
      grid-template-columns: auto auto auto; // ②
      background-color: #f7ef7e;
      padding: 3px;
    }
    .item {
      background-color: rgb(255, 255, 255);
      border: 1px solid rgb(0, 0, 0);
      padding: 20px;
      font-size: 30px;
      text-align: center;
    }
  </style>
</head>
```

Sol.

```
<body>  
  <div class="container">  
    <div class="item">1</div>  
    <div class="item">2</div>  
    <div class="item">3</div>  
    <div class="item">4</div>  
    <div class="item">5</div>  
    <div class="item">6</div>  
  </div>  
</body>  
</html>
```

fr 단위

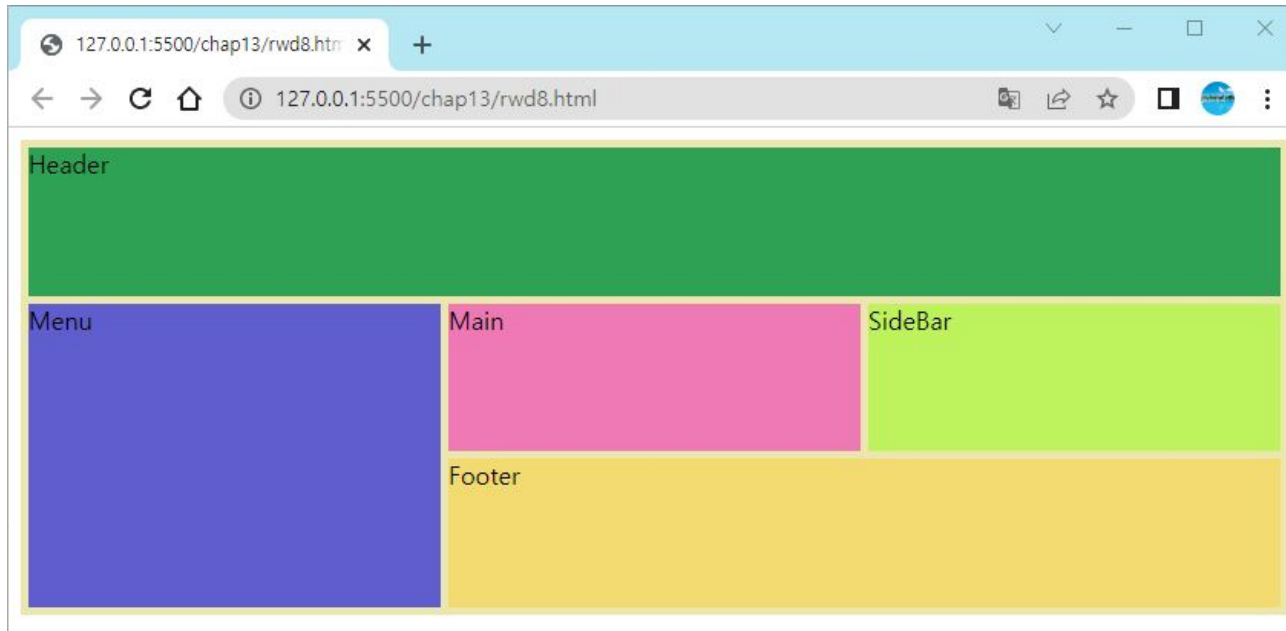
- 그리드 레이아웃에서 많이 사용하는 크기 단위에 **fr(fraction)**이 있다. 이것은 각 열의 너비를 상대적으로 정의할 때 편리하다.

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr; /* 1:1:1 */  
}
```

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 2fr 2fr; /* 1:2:2 */  
}
```

그리드 항목에 이름 붙여서 배치하기

- `grid-area` 속성을 이용하여 각 항목에 이름을 붙인 후에 컨테이너의 `grid-template-areas` 속성에 이름을 나열하는 것이다.



Sol.

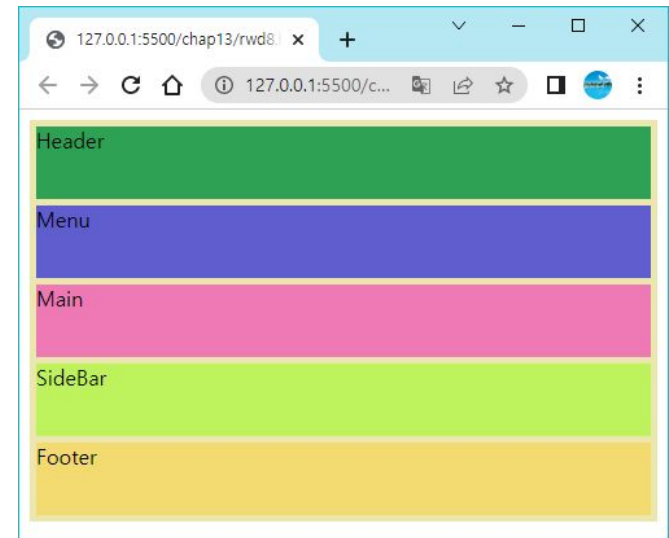
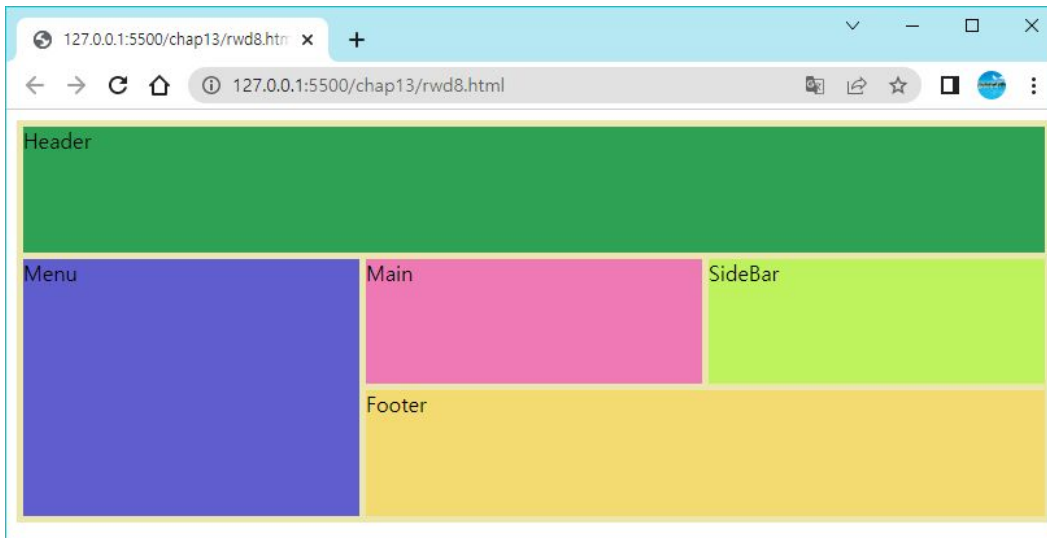
```
<!DOCTYPE html>
<html>
<head>
  <style>
    .item1 {
      grid-area: header;
      background-color: #2ea155;
    }
    .item2 {
      grid-area: menu;
      background-color: #605ece;
    }
    .item3 {
      grid-area: article;
      background-color: #ee79b4;
    }
    .item4 {
      grid-area: sidebar;
      background-color: #bff35e;
    }
    .item5 {
      grid-area: footer;
      background-color: #f3db72;
    }
  </style>
</head>
<body>
  <div class="item1">Header</div>
  <div class="item2">Menu</div>
  <div class="item3">Article</div>
  <div class="item4">Sidebar</div>
  <div class="item5">Footer</div>
</body>
</html>
```

Sol.

```
.container {  
  display: grid;  
  height: 300px;  
  grid-template-columns: 1fr 1fr 1fr;  
  grid-template-rows: 1fr 1fr 1fr;  
  grid-template-areas:  
    'header header header'  
    'menu article sidebar'  
    'menu footer footer';  
  gap: 5px;  
  background-color: #ece6af;  
  padding: 5px;  
}  
</style>  
</head>  
<body>  
  <div class="container">  
    <div class="item1">Header</div>  
    <div class="item2">Menu</div>  
    <div class="item3">Main</div>  
    <div class="item4">SideBar</div>  
    <div class="item5">Footer</div>  
  </div>  
</body>
```


grid 레이아웃을 이용한 반응형 웹디자인

- grid 레이아웃에서도 미디어 쿼리의 결과에 따라서 그리드의 열이나 행의 수를 변경할 수 있다. 예를 들어서 화면의 너비가 800픽셀보다 작으면 모든 요소들은 1열로 배치하도록 앞의 소스를 변경해보자.



Sol.

...

```
.container {  
  display: grid;  
  gap: 5px;  
  background-color: #ece6af;  
  grid-template-columns: 1fr;  
  grid-template-areas:  
    'header'  
    'menu'  
    'article'  
    'sidebar'  
    'footer';  
  height: 300px;  
  padding: 5px;  
}  
@media (min-width: 800px) {  
  .container {  
    grid-template-columns: 1fr 1fr 1fr;  
    grid-template-rows: 1fr 1fr 1fr;  
    grid-template-areas:  
      'header header header'  
      'menu article sidebar'  
      'menu footer footer';  
  }
```

웹 앱

- **HTML5**과 **CSS**, 자바스크립트를 이용하면 웬만한 애플리케이션의 기능은 흉내 낼 수 있다. 모바일 장치에서는 웹 페이지로 앱을 제작하기도 한다. 이것을 웹 앱(**Web App**)이라고 한다.
- 전통적인 프로그래밍 언어와 장치의 **SDK**를 이용하여 제작되는 앱을 네이티브 앱(**Native App**)이라고 한다.

Native App

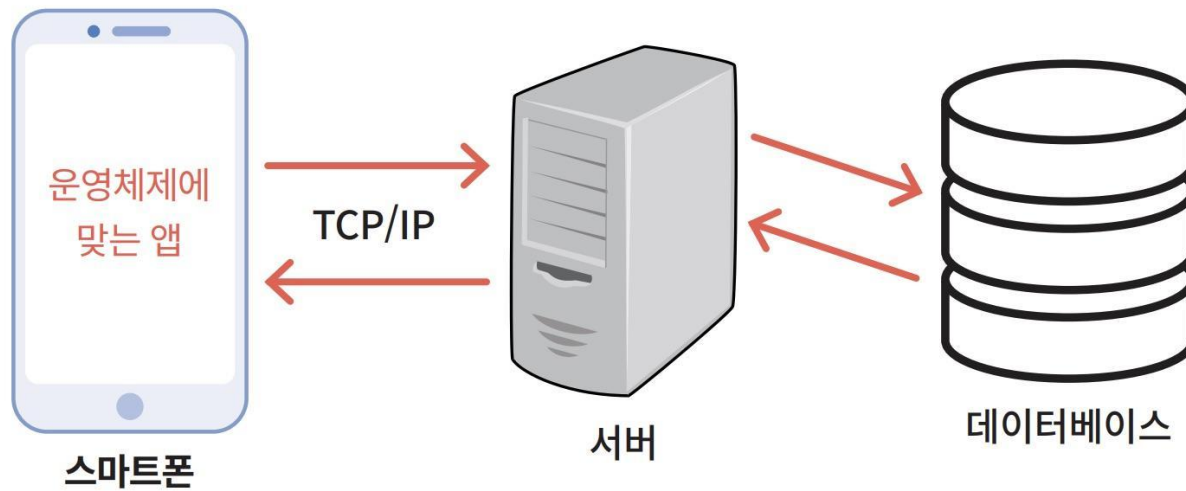


Web App



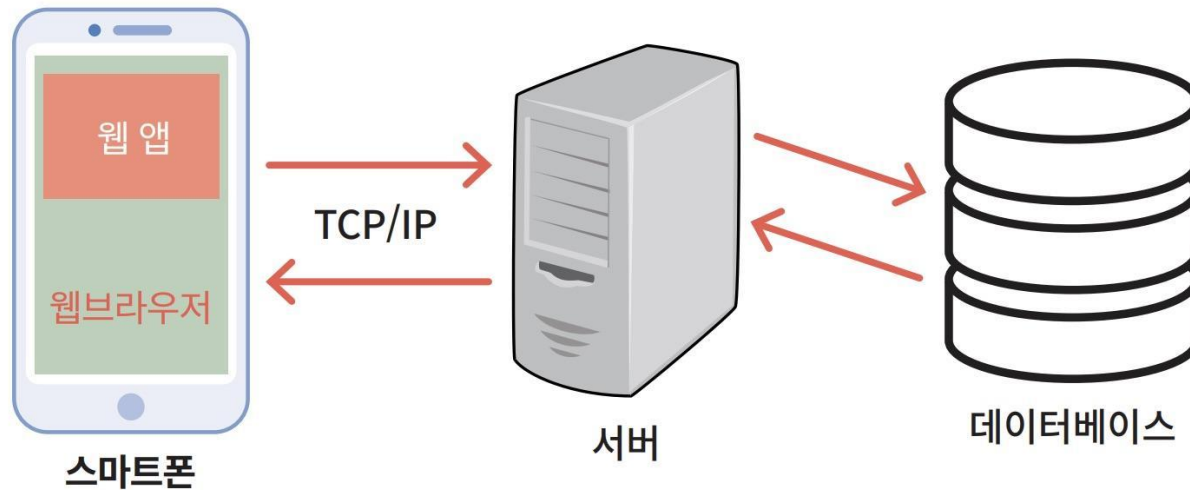
네이티브 앱

- 네이티브 앱은 모바일 장치 안에 설치되며 특정 모바일 운영체제용으로 개발된 프로그램이다. 이러한 앱은 **Apple App Store, Google Play Store** 등과 같은 앱 스토어에서 다운로드한 후에 설치하여서 사용할 수 있다.
- 각 운영체제에 맞는 프로그램을 작성하려면 특정 언어를 사용하여야 하고 특정 플랫폼의 프로그래밍 기술을 학습하여야 한다.



웹 앱

- 웹 앱은 **HTML** 및 **CSS**의 웹 페이지로 개발되고 대화형 부분은 자바스크립트 언어로 개발된다.
- 웹 앱은 웹 브라우징이 가능한 대부분의 장치에서 사용할 수 있으며 사용하는 운영체제에 의존하지 않다.



웹 앱 vs 네이티브 앱

	Native App.	Web App.
실행속도	빠름	일반적으로 느림
플랫폼	플랫폼마다 제작하여야 함	하나의 버전으로 모든 플랫폼에서 실행 가능
하드웨어 접근	GPS나 카메라에 접근할 수 있음	일반적으로는 하드웨어에 접근하는 것이 어려움
배포	앱 마켓을 통한 배포	배포할 필요가 없음
버전 업데이트	상당한 시간이 걸림	즉시 반영
오프라인시	사용 가능	약간의 기능 사용 가능



웹 앱의 단점

- 작은 화면 크기 - 일반적으로 모바일 장치의 화면은 데스크탑 컴퓨터 화면에 비하여 상당히 작다. 따라서 텍스트와 그래픽을 보기 어렵게 하거나 불가능하게 한다.
- 윈도우 부족 - 데스크탑 컴퓨터에서 한 번에 둘 이상의 창을 열 수 있는 기능은 멀티태스킹을 허용하고 이전 페이지로 쉽게 되돌릴 수 있다. 하지만 예전의 모바일 웹에서는 한 번에 한 페이지만 표시할 수 있었고 페이지는 원래 액세스한 순서대로만 볼 수 있었다.
- 속도 - 대부분의 모바일 장치에서 서비스 속도는 데스크탑에 비하여 느리다. 메모리와 **CPU**도 데스크탑에 느리다. 키보드와 마우스를 사용할 수 없고 터치만 사용할 수 있다.
- 고유기능 사용불가 - 카메라, 전화, 블루투스, **NFC** 등 디바이스의 고유기능을 사용할 수 없다.

하이브리드 앱

- 하이브리드앱의 내부 작동은 웹 앱과 유사하지만 네이티브 앱처럼 설치된다. 하이브리드앱은 모바일 장치 **API**에 액세스할 수 있으므로 카메라, 저장장치, GPS와 같은 리소스를 사용할 수 있다.

